

M05W04 - Optimizer

Pham Bao Long

November 23, 2024

1 Introduction

In this assignment, I aim to present and analyze fundamental optimization algorithms commonly used in deep learning. The complete implementation of the project is available on my [GitHub repository](#).

2 Implementation

2.1 Gradient Descent

Optimization function:

$$f(w_1, w_2) = 0.1w_1^2 + 2w_2^2$$

Gradient Descent Function:

$$W_t = W_{t-1} - \alpha \cdot \nabla f(W_{t-1})$$

Where:

- W_{t-1} : The previous weight or parameter being optimized.
- α : The learning rate, controlling the step size.
- $\nabla f(W)$: The gradient of the optimize function with respect to W .

Implementation:

1. Step 1: Compute the derivative of the optimization function.

The optimization function is given by:

$$f(w_1, w_2) = 0.1w_1^2 + 2w_2^2$$

The partial derivatives with respect to w_1 and w_2 are:

$$\frac{\partial f(w_1, w_2)}{\partial w_1} = 0.2w_1$$

$$\frac{\partial f(w_1, w_2)}{\partial w_2} = 4w_2$$

Combine into the Gradient Vector:

$$\nabla f(W_{t-1}) = \begin{bmatrix} \frac{\partial f}{\partial w_1} \\ \frac{\partial f}{\partial w_2} \end{bmatrix} = \begin{bmatrix} 0.2w_1 \\ 4w_2 \end{bmatrix}$$

However, the input is an array:

$$W_{t-1} = [w_1, w_2]$$

Then, the gradient is computed as:

$$\nabla f(W_{t-1}) = [w_1 \quad w_2] \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix} = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix}$$

For example,

$$\begin{aligned} W_{t-1} &= [-5 \quad -2] \\ \Rightarrow \nabla f(W_{t-1}) &= [-5 \quad -2] \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix} = [-1 \quad -8] \end{aligned}$$

2. Step 2: Use the Gradient Descent function to update the value of W .

Continuing the previous example, we apply the Gradient Descent function to the optimization problem.

Given:

- $W_{t-1} = \begin{bmatrix} -5 & -2 \end{bmatrix}$ (The previous weight or parameter being optimized)
- $\alpha = 0.4$ (The learning rate, controlling the step size)
- $\nabla f(W_{t-1}) = \begin{bmatrix} -1 & -8 \end{bmatrix}$ (The gradient of the optimization function with respect to W_{t-1})

The updated W_i is computed as:

$$W_t = W_{t-1} - \alpha \cdot \nabla f(W)$$

Substituting the values, we get:

$$W_t = \begin{bmatrix} -5 & -2 \end{bmatrix} - 0.4 \cdot \begin{bmatrix} -1 & -8 \end{bmatrix} = \begin{bmatrix} -4.6 & 1.2 \end{bmatrix}$$

3. Step 3: Repeat the process using the updated W .

Continuing with the previous example, the updated W_t is now redefined as W_{t-1} for the next iteration.

Given:

- $W_{t-1} = \begin{bmatrix} -4.6 & 1.2 \end{bmatrix}$ (The previous weight or parameter being optimized)
- $\alpha = 0.4$ (The learning rate, controlling the step size)

The gradient now is computed:

$$\begin{aligned} \nabla f(W_{t-1}) &= W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix} \\ &= \begin{bmatrix} -4.6 & 1.2 \end{bmatrix} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix} = \begin{bmatrix} -0.92 & 4.8 \end{bmatrix} \end{aligned}$$

The updated value of W_i is:

$$\begin{aligned} W_t &= W_{t-1} - \alpha \cdot \nabla f(W_{t-1}) \\ W_t &= \begin{bmatrix} -4.6 & 1.2 \end{bmatrix} - 0.01 \cdot \begin{bmatrix} -0.92 & 4.8 \end{bmatrix} = \begin{bmatrix} -4.232 & -0.72 \end{bmatrix} \end{aligned}$$

Notation: This process can be repeated iteratively until the optimal W_i is obtained.

Here is the reference Python code for the above calculation. It implements the process over 30 epochs and returns a list of W values for every update:

```
1 import numpy as np
2
3 def df_W(W):
4     """
5     Calculate the gradient of the function f(w1, w2) = 0.1w1^2 + 2w2^2.
6     """
7     # Coefficient matrix for the diagonal terms in the gradient
8     coefficient = np.diag([0.2, 4])
9     # Gradient calculation
10    dw = np.dot(W, coefficient)
11    return dw
12
13 def sgd(W, dW, lr):
14     """
15     Update weights using Stochastic Gradient Descent (SGD).
16     """
17    W = W - lr * dW
18    return W
```

Listing 1: Python Code for Gradient Descent

```

1  def train_pl(optimizer, lr, epochs):
2      """
3      Train the model over a given number of epochs,
4      using the provided optimizer and learning rate.
5      """
6      # Initial weights
7      W = np.array([-5, -2], dtype=np.float32)
8      results = [W] # List to store W over all epochs
9
10     for epoch in range(epochs):
11         # Compute the gradient
12         dW = df_W(W)
13         # Update weights using the optimizer
14         W = optimizer(W, dW, lr)
15         # Store the updated weights
16         results.append(W)
17
18     return results
19
20 # Training process
21 weights_history = train_pl(sgd, lr=0.4, epochs=30)
22
23 # Print the results
24 for epoch, W in enumerate(weights_history):
25     print(f"Epoch {epoch}: W = {W}")

```

Listing 2: Python Code for Gradient Descent

2.2 Momentum-based Gradient Descent

Optimization function:

$$f(w_1, w_2) = 0.1w_1^2 + 2w_2^2$$

$$\Rightarrow \nabla f(W_{t-1}) = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix}$$

Momentum update rule:

$$V_t = \beta \cdot V_{t-1} + (1 - \beta) \cdot \nabla f(W_{t-1})$$

Momentum-based Gradient Descent function:

$$W_t = W_{t-1} - \alpha \cdot V_t$$

Implementation:

1. **Step 1: Compute the derivative of the optimization function.**

Given:

- $W_{t-1} = [-5 \quad -2]$ (The previous weight or parameter being optimized)

The gradient is computed as:

$$\nabla f(W_{t-1}) = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix}$$

Substituting W_{t-1} :

$$\Rightarrow \nabla f(W_{t-1}) = [-5 \quad -2] \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix} = [-1 \quad -8]$$

2. Step 2: Compute V_t (Velocity update):

Given:

- β : The momentum rate, which determines the influence of the previous velocity on the current update.
- V_{t-1} : The initial velocity, set to zero for all dimensions.
- $\nabla f(W_{t-1})$: The gradient of the optimization function with respect to W_{t-1} .

The velocity is computed using the momentum update rule as follows:

$$V_t = \beta \cdot V_{t-1} + (1 - \beta) \cdot \nabla f(W_{t-1}),$$

where $\beta = 0.5$, $V_{t-1} = [0 \ 0]$, and $\nabla f(W_{t-1}) = [-1 \ -8]$. Substituting these values:

$$V_t = 0.5 \cdot [0 \ 0] + (1 - 0.5) \cdot [-1 \ -8]$$

$$\implies V_t = [0 \ 0] + 0.5 \cdot [-1 \ -8] = [-0.5 \ -4.0].$$

3. Step 3: Update Weights (Momentum-Based Gradient Descent):

Given:

- W_{t-1} : The initial weights.
- α : The learning rate, which controls the step size.
- V_t : The updated velocity.

The weights are updated using the momentum-based gradient descent formula:

$$W_t = W_{t-1} - \alpha \cdot V_t,$$

where $W_{t-1} = [-5 \ -2]$, $\alpha = 0.6$, and $V_t = [-0.5 \ 4.0]$. Substituting these values:

$$W_t = [-5 \ -2] - 0.6 \cdot [-0.5 \ 4.0] = [-4.7 \ 0.4]$$

4. Step 4: Repeat the Process with Updated W :

Continuing from the first update, W_t and V_t is now redefined as W_{t-1} and V_{t-1} respectively for the next iteration.

Given:

- W_{t-1} : The previous updated weight vector.
- V_{t-1} : The velocity from the previous iteration.
- α : The consistent learning rate used for this iteration.
- β : The consistent momentum coefficient.

First, compute the gradient:

$$\nabla f(W_{t-1}) = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix},$$

where $W_{t-1} = [-4.7 \ 0.4]$. Substituting values:

$$\nabla f(W_{t-1}) = [-4.7 \ 0.4] \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix} = [-0.94 \ 1.6].$$

Next, compute the velocity:

$$V_t = \beta \cdot V_{t-1} + (1 - \beta) \cdot \nabla f(W_{t-1}),$$

where $\beta = 0.5$, $V_{t-1} = [-0.5 \quad -4.0]$, and $\nabla f(W_{t-1}) = [-0.94 \quad 1.6]$. Substituting these values:

$$V_t = 0.5 \cdot [-0.5 \quad -4.0] + (1 - 0.5) \cdot [-0.94 \quad 1.6]$$

$$\Rightarrow V_t = [-0.25 \quad -2] + [-0.47 \quad 0.8] = [-0.72 \quad -1.2].$$

Finally, update the weights:

$$W_t = W_{t-1} - \alpha \cdot V_t,$$

where $W_{t-1} = [-4.7 \quad 0.4]$, $\alpha = 0.6$, and $V_t = [-0.72 \quad -1.2]$. Substituting these values:

$$W_t = [-4.7 \quad 0.4] - 0.6 \cdot [-0.72 \quad -1.2]$$

$$\Rightarrow W_t = [-4.7 \quad 0.4] + [0.432 \quad 0.72] = [-4.268 \quad 1.12].$$

Notation: This process can be repeated iteratively until the optimal W_i is obtained.

Here is the reference Python code for the above calculation. It implements the process over 30 epochs and returns a list of W values for every update:

```
1 import numpy as np
2
3 def df_W(W):
4     """
5     Compute the gradient of the function f with respect to W.
6     """
7     return np.dot(W, np.diag([0.2, 4]))
8
9 def velocity(dW, V, momentum):
10    """
11    Update the velocity using the momentum term.
12    """
13    return momentum * V + (1 - momentum) * dW
14
15 def sgd_momentum(W, lr, Vel):
16    """
17    Update weights using momentum-based gradient descent.
18    """
19    return W - lr * Vel
```

Listing 3: Python Code for Momentum-based Gradient Descent

```

1 def train_pl(lr, momentum, epochs):
2     """
3     Train a model using momentum-based stochastic gradient descent.
4
5     Parameters:
6         lr (float): Learning rate.
7         momentum (float): Momentum coefficient.
8         epochs (int): Number of epochs to train.
9
10    Returns:
11        list: A list of weights at each epoch.
12    """
13    # Initialize weights and velocity
14    W = np.array([-5, -2], dtype=float)
15    V = np.zeros_like(W)
16    results = [W.copy()]
17
18    for epoch in range(epochs):
19        # Compute the gradient
20        dW = df_W(W)
21
22        # Update the velocity
23        V = velocity(dW, V, momentum)
24
25        # Update weights
26        W = sgd_momentum(W=W, lr=lr, Vel=V)
27        results.append(W.copy())
28
29        # Print progress
30        print(f"Epoch {epoch + 1}:")
31        print(f"    Gradient (dW): {np.round(dW, 5)}")
32        print(f"    Velocity (V): {np.round(V, 5)}")
33        print(f"    Updated Weights (W): {np.round(W, 5)}\n")
34
35    return results
36
37 # Run the training loop
38 train_pl(lr=0.6, momentum=0.5, epochs=30)

```

Listing 4: Python Code for Momentum-based Gradient Descent

2.3 Root Mean Square Propagation (RMSProp)

Optimization function:

$$f(w_1, w_2) = 0.1w_1^2 + 2w_2^2$$

$$\Rightarrow \nabla f(W_{t-1}) = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix}$$

Exponential Moving Average function (EMA):

$$S_t = \gamma \cdot S_{t-1} + (1 - \gamma) \cdot (\nabla f(W_{t-1}))^2,$$

RMSProp optimizer:

$$W_t = W_{t-1} - \alpha \cdot \frac{\nabla f(W_{t-1})}{\sqrt{S_t + \epsilon}}$$

Implementation:

1. Step 1: Compute the gradient (Derivative of optimization function):

Given:

- W_{t-1} : The initial weights being optimized.

The gradient is computed as:

$$\nabla f(W_{t-1}) = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix}$$

where $W_{t-1} = \begin{bmatrix} -5 & -2 \end{bmatrix}$:

$$\Rightarrow \nabla f(W_{t-1}) = \begin{bmatrix} -5 & -2 \end{bmatrix} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix} = \begin{bmatrix} -1 & -8 \end{bmatrix}$$

2. Step 2: Calculate the Exponential Moving Average (EMA):

Given:

- S_{t-1} : The initial value of the EMA, set to neutral.
- γ : The decay rate, determining the weight of past values.
- $\nabla f(W_{t-1})$: The gradient of the optimization function with respect to the initial weights.

The EMA is computed as:

$$S_t = \gamma \cdot S_{t-1} + (1 - \gamma) \cdot (\nabla f(W_{t-1}))^2$$

where $S_{t-1} = \begin{bmatrix} 0 & 0 \end{bmatrix}$, $\gamma = 0.9$, and $\nabla f(W_{t-1}) = \begin{bmatrix} -1 & -8 \end{bmatrix}$. Substituting these values:

$$S_t = 0.9 \cdot \begin{bmatrix} 0 & 0 \end{bmatrix} + (1 - 0.9) \cdot \begin{bmatrix} -1 & -8 \end{bmatrix}^2$$

$$\Rightarrow S_t = \begin{bmatrix} 0 & 0 \end{bmatrix} + \begin{bmatrix} 0.1 & 6.4 \end{bmatrix} = \begin{bmatrix} 0.1 & 6.4 \end{bmatrix}.$$

3. Step 3: Update the weights using the RMSProp optimizer:

Given:

- W_{t-1} : The initial weights.
- $\nabla f(W_{t-1})$: The gradient of the optimization function with respect to W_{t-1} .
- α : The learning rate of the optimizer.
- S_t : The updated exponential moving average of squared gradients.
- ϵ : A small positive constant to prevent division by zero.

The updated weights are computed as:

$$W_t = W_{t-1} - \alpha \cdot \frac{\nabla f(W_{t-1})}{\sqrt{S_t + \epsilon}}$$

where $W_{t-1} = \begin{bmatrix} -5 & -2 \end{bmatrix}$, $\nabla f(W_{t-1}) = \begin{bmatrix} -1 & -8 \end{bmatrix}$, $\alpha = 0.3$, $S_t = \begin{bmatrix} 0.1 & 6.4 \end{bmatrix}$, and $\epsilon = 0.000001$. Substituting these values:

$$W_t = \begin{bmatrix} -5 & -2 \end{bmatrix} - 0.3 \cdot \frac{\begin{bmatrix} -1 & -8 \end{bmatrix}}{\sqrt{\begin{bmatrix} 0.1 & 6.4 \end{bmatrix} + 0.000001}}$$

$$\Rightarrow W_t = \begin{bmatrix} -4.05132145 & -1.05131678 \end{bmatrix}.$$

4. Step 4: Repeat the Process with the Updated Weights:

At this step, the newly updated W_t and S_t are redefined as W_{t-1} and S_{t-1} , respectively, to prepare for the next iteration.

Given:

- W_{t-1} : The previous weight vector after the last update.
- S_{t-1} : The exponential moving average (EMA) from the previous iteration.
- α : The consistent learning rate for the optimizer.
- γ : The decay rate of the EMA.
- ϵ : A small positive constant to prevent division by zero.

First, compute the gradient:

$$\nabla f(W_{t-1}) = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix},$$

where $W_{t-1} = [-4.05132145 \quad -1.05131678]$. Substituting the values:

$$\nabla f(W_{t-1}) = [-4.05132145 \quad -1.05131678] \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix} = [-0.81026429 \quad -4.2052671].$$

Next, compute the updated EMA:

$$S_t = \gamma \cdot S_{t-1} + (1 - \gamma) \cdot (\nabla f(W_{t-1}))^2,$$

where $S_{t-1} = [0.1 \quad 6.4]$, $\gamma = 0.9$, and $\nabla f(W_{t-1}) = [-0.81026429 \quad -4.2052671]$. Substituting these values:

$$S_t = 0.9 \cdot [0.1 \quad 6.4] + (1 - 0.9) \cdot [-0.81026429 \quad -4.2052671]^2 = [0.15565282 \quad 7.52842714].$$

Finally, compute the updated weight:

$$W_t = W_{t-1} - \alpha \cdot \frac{\nabla f(W_{t-1})}{\sqrt{S_t + \epsilon}},$$

where $W_{t-1} = [-4.05132145 \quad -1.05131678]$, $\nabla f(W_{t-1}) = [-0.81026429 \quad -4.2052671]$, $\alpha = 0.3$, $S_t = [0.15565282 \quad 7.52842714]$, and $\epsilon = 0.001$. Substituting the values:

$$W_t = [-4.05132145 \quad -1.05131678] - 0.3 \cdot \frac{[-0.81026429 \quad -4.2052671]}{\sqrt{[0.15565282 \quad 7.52842714] + 0.000001}} = [-4.46374 \quad -1.47391].$$

$$\Rightarrow W_t = [-3.43519754 \quad -0.59152343]$$

Notation: This process can be repeated iteratively until the optimal W_i is obtained.

Here is the reference Python code for the above calculation. It implements the process over 30 epochs and returns a list of W values for every update:

```

1 import numpy as np
2
3 def compute_gradient(W):
4     """
5     Compute the gradient of the objective function with respect to W.
6     Here, we assume a simple linear function with a diagonal matrix for simplicity
7     """
8     return np.dot(W, np.diag([0.2, 4]))

```

Listing 5: Python Code for Root Mean Square Propagation (RMSProp)


```

1 def update_ema(S, decay_rate, gradient_squared):
2     """
3     Update the Exponential Moving Average (EMA) of squared gradients.
4
5     Parameters:
6         S (ndarray): The previous EMA of squared gradients.
7         decay_rate (float): The decay factor for EMA.
8         gradient_squared (ndarray): The square of the gradients.
9
10    Returns:
11        ndarray: The updated EMA of squared gradients.
12    """
13    return decay_rate * S + (1 - decay_rate) * gradient_squared
14
15 def rmsprop_update(W, lr, gradient, ema, epsilon=0.000001):
16     """
17     Perform a single update using the RMSProp optimizer.
18
19     Parameters:
20         W (ndarray): The current weights.
21         lr (float): The learning rate.
22         gradient (ndarray): The gradient of the objective function.
23         ema (ndarray): The Exponential Moving Average of squared gradients.
24         epsilon (float): A small constant to prevent division by zero.
25
26     Returns:
27         ndarray: The updated weights.
28     """
29    return W - lr * (gradient / np.sqrt(ema + epsilon))
30
31 def train_rmsprop(lr, decay_rate, epochs):
32     """
33     Train a simple model using the RMSProp optimization algorithm.
34
35     Parameters:
36         lr (float): The learning rate for RMSProp.
37         decay_rate (float): The decay rate for EMA.
38         epochs (int): The number of training iterations (epochs).
39     """
40    # Initialize weights (W) and EMA (S) for RMSProp
41    W = np.array([-5, -2])
42    S = np.zeros_like(W) # EMA initialized to zero
43    results = [W.copy()]
44
45    # Training loop
46    for epoch in range(epochs):
47        # Compute gradient
48        gradient = compute_gradient(W)
49
50        # Update EMA with squared gradients
51        S = update_ema(S, decay_rate, gradient**2)
52
53        # Update weights using RMSProp
54        W = rmsprop_update(W, lr, gradient, S)
55
56        # Save the updated weights for tracking
57        results.append(W.copy())
58
59        # Print progress (optional for monitoring training)
60        print(f"Epoch {epoch + 1}:")
61        print(f"    Gradient (dW): {gradient}")
62        print(f"    EMA (S): {S}")
63        print(f"    Updated Weights (W): {W}\n")
64
65    return results
66
67 # Call the training function with parameters
68 train_rmsprop(lr=0.3, decay_rate=0.9, epochs=30)

```

Listing 6: Python Code for Root Mean Square Propagation (RMSProp)

2.4 Adaptive Moment Estimation (Adam)

Optimization Function:

$$f(w_1, w_2) = 0.1w_1^2 + 2w_2^2 \\ \Rightarrow \nabla f(W_{t-1}) = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix}$$

Momentum Update Rule:

$$V_t = \beta \cdot V_{t-1} + (1 - \beta) \cdot \nabla f(W_{t-1})$$

Exponential Moving Average (EMA):

$$S_t = \gamma \cdot S_{t-1} + (1 - \gamma) \cdot (\nabla f(W_{t-1}))^2$$

Bias Correction for Velocity (Momentum Update Rule):

$$V_t^{\text{corr}} = \frac{V_t}{1 - \beta^t}$$

Bias Correction for Moving Average (EMA):

$$S_t^{\text{corr}} = \frac{S_t}{1 - \gamma^t}$$

Adaptive Moment Estimation Update rule:

$$W_t = W_{t-1} - \alpha \cdot \frac{V_t^{\text{corr}}}{\sqrt{S_t^{\text{corr}} + \epsilon}}$$

Implementation:

1. **Step 1: Compute the gradient (Derivative of optimization function):**

Given:

- W_{t-1} : The initial weights being optimized.

The gradient is computed as:

$$\nabla f(W_{t-1}) = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix}$$

where $W_{t-1} = \begin{bmatrix} -5 & -2 \end{bmatrix}$:

$$\Rightarrow \nabla f(W_{t-1}) = \begin{bmatrix} -5 & -2 \end{bmatrix} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix} = \begin{bmatrix} -1 & -8 \end{bmatrix}$$

2. **Step 2: Compute V_t (Velocity update):**

Given:

- β : The momentum rate, which determines the influence of the previous velocity on the current update.
- V_{t-1} : The initial velocity, set to zero for all dimensions.
- $\nabla f(W_{t-1})$: The gradient of the optimization function with respect to W_{t-1} .

The velocity is computed using the momentum update rule as follows:

$$V_t = \beta \cdot V_{t-1} + (1 - \beta) \cdot \nabla f(W_{t-1}),$$

where $\beta = 0.9$, $V_{t-1} = \begin{bmatrix} 0 & 0 \end{bmatrix}$, and $\nabla f(W_{t-1}) = \begin{bmatrix} -1 & -8 \end{bmatrix}$. Substituting these values:

$$V_t = 0.9 \cdot \begin{bmatrix} 0 & 0 \end{bmatrix} + (1 - 0.9) \cdot \begin{bmatrix} -1 & -8 \end{bmatrix}$$

$$\Rightarrow V_t = \begin{bmatrix} 0 & 0 \end{bmatrix} + 0.1 \cdot \begin{bmatrix} -1 & -8 \end{bmatrix} = \begin{bmatrix} -0.1 & -0.8 \end{bmatrix}.$$

3. Step 3: Calculate the Exponential Moving Average (EMA):

Given:

- S_{t-1} : The initial value of the EMA, set to neutral.
- γ : The decay rate, determining the weight of past values.
- $\nabla f(W_{t-1})$: The gradient of the optimization function with respect to the initial weights.

The EMA is computed as:

$$S_t = \gamma \cdot S_{t-1} + (1 - \gamma) \cdot (\nabla f(W_{t-1}))^2$$

where $S_{t-1} = [0 \ 0]$, $\gamma = 0.999$, and $\nabla f(W_{t-1}) = [-1 \ -8]$. Substituting these values:

$$S_t = 0.999 \cdot [0 \ 0] + (1 - 0.999) \cdot [-1 \ -8]^2$$

$$\Rightarrow S_t = [0 \ 0] + [0.001 \ 0.064] = [0.001 \ 0.064] .$$

4. Step 4: Calculate Bias Correction of Velocity and EMA:

a. Bias-Correction of Velocity:

Given:

- V_t : The updated velocity used for correction.
- β : The momentum coefficient used for this correction.
- t : The index of the iteration.

The bias correction of velocity is computed as:

$$V_t^{\text{corr}} = \frac{V_t}{1 - \beta^t}$$

where $V_t = [-0.1 \ -0.8]$, $\beta = 0.9$, and $t = 1$. Substituting these values:

$$V_t^{\text{corr}} = \frac{[-0.1 \ -0.8]}{1 - 0.9^1} = [-1.0 \ -8.0]$$

b. Bias-Correction of Moving Average:

Given:

- S_t : The updated moving average used for this correction.
- γ : The decay rate used for this correction.
- t : The index of the iteration.

The bias correction of the moving average is computed as:

$$S_t^{\text{corr}} = \frac{S_t}{1 - \gamma^t}$$

where $S_t = [0.001 \ 0.064]$, $\gamma = 0.999$, and $t = 1$. Substituting these values:

$$S_t^{\text{corr}} = \frac{[0.001 \ 0.064]}{1 - 0.999^1} = [1.0 \ 64.0] .$$

5. Step 5: Update the weights using the Adam optimizer: Given:

- W_{t-1} : The initial weights.
- V_t^{corr} : The Bias correction of the Velocity.
- S_t^{corr} : The Bias correction of the Moving Average.
- α : The consistent learning rate.
- ϵ : The small positive constant

The weights is updated as:

$$W_t = W_{t-1} - \alpha \cdot \frac{V_t^{\text{corr}}}{\sqrt{S_t^{\text{corr}} + \epsilon}}$$

where $W_{t-1} = \begin{bmatrix} -5 & -2 \end{bmatrix}$, $V_t^{\text{corr}} = \begin{bmatrix} -1.0 & -8.0 \end{bmatrix}$, $S_t^{\text{corr}} = \begin{bmatrix} 1.0 & 64.0 \end{bmatrix}$, $\alpha = 0.2$, and $\epsilon = 0.000001$. Substituting those values:

$$W_t = \begin{bmatrix} -5 & -2 \end{bmatrix} - 0.2 \cdot \frac{\begin{bmatrix} -1.0 & -8.0 \end{bmatrix}}{\sqrt{\begin{bmatrix} 1.0 & 64.0 \end{bmatrix} + 0.000001}} = \begin{bmatrix} -4.8000002 & -1.80000002 \end{bmatrix}$$

6. Step 6: Repeat the Process with the Updated Weights:

For the second iteration, W_t , V_t , and S_t are redefined as W_{t-1} , V_{t-1} , and S_{t-1} . Especially, t is set to be 2. **Given:**

- W_{t-1} : The previous weight vector after the last update.
- V_{t-1} : The velocity from the previous iteration.
- β : The consistent momentum coefficient.
- S_{t-1} : The exponential moving average (EMA) from the previous iteration.
- γ : The decay rate of the EMA.
- α : The consistent learning rate for the optimizer.
- ϵ : A small positive constant to prevent division by zero.

Firstly, compute the gradient:

$$\nabla f(W_{t-1}) = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix}$$

where $W_{t-1} = \begin{bmatrix} -4.8000002 & -1.80000002 \end{bmatrix}$:

$$\Rightarrow \nabla f(W_{t-1}) = \begin{bmatrix} -4.8000002 & -1.80000002 \end{bmatrix} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix} = \begin{bmatrix} -0.96000004 & -7.2000001 \end{bmatrix}.$$

Secondly, compute Velocity and Moving Average:

Velocity is computed as:

$$V_t = \beta \cdot V_{t-1} + (1 - \beta) \cdot \nabla f(W_{t-1}),$$

where $\beta = 0.9$, $V_{t-1} = \begin{bmatrix} -0.1 & -0.8 \end{bmatrix}$, and $\nabla f(W_{t-1}) = \begin{bmatrix} -0.96000004 & -7.2000001 \end{bmatrix}$. Substituting these values:

$$V_t = 0.9 \cdot \begin{bmatrix} -0.1 & -0.8 \end{bmatrix} + (1 - 0.9) \cdot \begin{bmatrix} -0.96000004 & -7.2000001 \end{bmatrix} = \begin{bmatrix} -0.186 & -1.44000001 \end{bmatrix}$$

Moving Average is computed as:

$$S_t = \gamma \cdot S_{t-1} + (1 - \gamma) \cdot (\nabla f(W_{t-1}))^2$$

where $S_{t-1} = \begin{bmatrix} 0.001 & 0.064 \end{bmatrix}$, $\gamma = 0.999$, and $\nabla f(W_{t-1}) = \begin{bmatrix} -0.96000004 & -7.2000001 \end{bmatrix}$. Substituting these values:

$$S_t = 0.999 \cdot \begin{bmatrix} 0.001 & 0.064 \end{bmatrix} + (1 - 0.999) \cdot \begin{bmatrix} -0.96000004 & -7.2000001 \end{bmatrix}^2 = \begin{bmatrix} 0.0019206 & 0.115776 \end{bmatrix}$$

Thirdly, compute bias correction of velocity and moving average:

Bias correction of velocity:

$$V_t^{\text{corr}} = \frac{V_t}{1 - \beta^t}$$

where $V_t = [-0.186 \quad -1.44000001]$, $\beta = 0.9$, and $t = 2$. Substituting these values:

$$V_t^{\text{corr}} = \frac{[-0.186 \quad -1.44000001]}{1 - 0.9^2} = [-0.97894739 \quad -7.57894742]$$

Bias correction of Moving Average:

$$S_t^{\text{corr}} = \frac{S_t}{1 - \gamma^t}$$

where $S_t = [0.0019206 \quad 0.115776]$, $\gamma = 0.999$, and $t = 2$. Substituting these values:

$$S_t^{\text{corr}} = \frac{[0.0019206 \quad 0.115776]}{1 - 0.999^2} = [0.96078043 \quad 57.9169592].$$

Finally, update new weights:

$$W_t = W_{t-1} - \alpha \cdot \frac{V_t^{\text{corr}}}{\sqrt{S_t^{\text{corr}} + \epsilon}}$$

where $W_{t-1} = [-4.8000002 \quad -1.80000002]$, $V_t^{\text{corr}} = [-0.97894739 \quad -7.57894742]$, $S_t^{\text{corr}} = [0.96078043 \quad 57.9169592]$, $\alpha = 0.2$, and $\epsilon = 0.000001$. Substituting those values:

$$W_t = [-4.8000002 \quad -1.80000002] - 0.2 \cdot \frac{[-0.97894739 \quad -7.57894742]}{\sqrt{[0.96078043 \quad 57.9169592] + 0.000001}} = [-4.60025478 \quad -1.60082451]$$

Notation: This process can be repeated iteratively until the optimal W_i is obtained.

Here is the reference Python code for the above calculation. It implements the process over 30 epochs and returns a list of W values for every update:

```

1 import numpy as np
2
3 def df_W(W):
4     """Compute the gradient of W."""
5     return np.dot(W, np.diag([0.2, 4]))
6
7 def velocity(V, momentum, dW):
8     """Update the velocity with momentum."""
9     V = momentum * V + (1 - momentum) * dW
10    return V
11
12 def EMA(S, decay_rate, dW):
13     """Compute the Exponential Moving Average (EMA) of the gradient squares."""
14     S = decay_rate * S + (1 - decay_rate) * (dW ** 2)
15     return S
16
17 def V_corr(V, momentum, epoch):
18     """Apply bias correction to the velocity."""
19     V_corr = V / (1 - momentum ** epoch)
20     return V_corr
21
22 def S_corr(S, decay_rate, epoch):
23     """Apply bias correction to the EMA."""
24     S_corr = S / (1 - decay_rate ** epoch)
25     return S_corr

```

Listing 7: Python Code for Adaptive Moment Estimate Optimizer (Adam)

```

1 def Adam(W, lr, V_corr, S_corr, epsilon=0.000001):
2     """Update weights using the Adam optimizer."""
3     W = W - lr * (V_corr / (np.sqrt(S_corr) + epsilon))
4     return W
5
6 def train_pl(lr, momentum, decay_rate, epochs):
7     """Train the model with Adam optimizer."""
8     # Initial weights, EMA, and velocity
9     W = np.array([-5, -2])
10    S = np.zeros_like(W)
11    V = np.zeros_like(W)
12    result = [W]
13
14    # Training loop
15    for epoch in range(1, epochs + 1):
16        # Compute gradient
17        dW = df_W(W)
18
19        # Update velocity (momentum)
20        V = velocity(V, momentum, dW)
21
22        # Update EMA of squared gradients
23        S = EMA(S, decay_rate, dW)
24
25        # Apply bias correction
26        v_corr = V_corr(V, momentum, epoch)
27        s_corr = S_corr(S, decay_rate, epoch)
28
29        # Update weights using Adam optimizer
30        W = Adam(W, lr, v_corr, s_corr)
31
32        # Store the updated weights
33        result.append(W.copy())
34
35        # Print progress for monitoring
36        print(f"Epoch {epoch}:")
37        print(f"    Gradient (dW): {dW}")
38        print(f"    Velocity (V): {V}")
39        print(f"    EMA (S): {S}")
40        print(f"    Velocity_corr (V_corr): {v_corr}")
41        print(f"    EMA_corr (S_corr): {s_corr}")
42        print(f"    Updated Weights (W): {W}\n")
43
44    return result
45
46 # Run the training for 2 epochs with the specified parameters
47 train_pl(lr=0.2, momentum=0.9, decay_rate=0.999, epochs=30)

```

Listing 8: Python Code for Adaptive Moment Estimate Optimizer (Adam)